

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 1: Intro, Number Representations

Instructor: Justin Yokota

Agenda

- Introduction
 - Course Logistics
- Class Overview
- Binary
 - Binary and Hexadecimal
 - Representing Data using Binary
 - Binary Mathematics

Agenda

- **Introduction**
 - Course Logistics
- **Class Overview**
- **Binary**
 - Binary and Hexadecimal
 - Representing Data using Binary
 - Binary Mathematics

Justin (he/him)

- Lecturer since 2022
- Before that, taught 61C as a TA
- 13th time teaching 61C (5th time as lecturer)
- Interests: Puzzles, Crosswords, Celeste, MTG



We also have amazing TAs and tutors!!!

They're going to be the ones helping you in discussions,
labs, exam prep sections, and office hours!

Head TA: Ariana Abel

TAs: Adelina Chau, Dave Deriso, Enoch Chen, Jose Guajardo, Joshua Xiong, Kabeer Nayyar, Sneha Muppalla

Tutors: Alex Simpson, Alyssa Li, Kaleb Lee, William Li, Yolanda Wei

Communication

- Ed: <https://edstem.org>
 - Discussion forum for Q&A and announcements
 - For private matters, you can make a private post
 - We're going to restrict the use of private posts for debugging purposes this semester - come to OH first
- Course website: <https://cs61c.org>
 - Course schedule, lecture slides, assigned readings, and other resources are all posted here
- Email: cs61c@berkeley.edu
 - Ed response times will generally be faster, but email may be useful for private matters

Course Structure: Lectures

- You made it here, congrats!
- Monday through Thursday, 2:00-3:30 PM PT
- Lecture attendance is not taken
- All lectures will be recorded and posted

Course Structure: Discussions

- Mondays and Wednesdays
- Smaller, 1-hour long sections led by a TA to practice course content
- Monday and Wednesday sections will cover different content
- You can attend any discussion section you want
- Discussion 1 will be released this week and there will be no sections
 - We will release discussion recordings from a previous semester.
- Synchronous discussions will start next Monday (discussion 2)!
- Schedule will be available at <https://cs61c.org/su26/calendar/>

Course Structure: Labs

- Tuesdays and Thursdays
- 2 hours long, with 30 min–1 hour conceptual mini-lecture from a TA, and remaining time office hour style Q&A
- Only lab questions in lab!
- Lab 0 will be released this week and there will be no lab sections since it is mostly setup.
 - Please come to office hours or post on the Ed if you need help!
- Synchronous labs will start next Tuesday (lab 1)!
- Schedule will be available at <https://cs61c.org/su26/calendar/>

Course Structure: Projects

- 3 Projects; each ~2 weeks
 - Project 1: snek: get comfortable coding in C and using debugging tools!
 - Project 2: CS61Classify: code in assembly language (RISC-V)!
 - Project 3: CS61CPU: build a working CPU!
- Note: We usually provide increased support for projects (in the form of increased staffing) leading up to the posted, original deadline.
 - As a result, continuing to work on a project after the original deadline may lead to slower response times on Ed/longer wait times in OH. Staff will actively prioritize assignments whose deadlines have not passed.

Course Structure: Office Hours/Ed

- OH

- Space to ask questions about content, get help with projects, raise concerns with the course, etc. with staff member
- OH schedule will be available at <https://cs61c.org/su26/calendar/>
- OH queue at <https://oh.cs61c.org>
- OH will start Thursday
- Most OH will be in person, but we will have a few hours of remote OH every week

- Ed

- Space to ask questions about content & get help with projects in a **collaborative** setting
- We will be providing limited private Ed support this summer
 - However, public posts (and discussion) are still highly encouraged!
- We will only answer **private** Ed posts if you go to OH and a staff member told you to make an Ed post
 - **Go to OH!!!** It's so much better in terms of collaboration

Course Structure: Exams

- Quest: CBTF-based, July 13-15
- Midterm: Thursday, July 23, 7:00–9:00 PM PT
- Final: Tuesday, August 12, time TBD
- All exams will be in person
- Clobber
 - If you score better on the final, we will use the [z-score](#) of your final exam score to replace your midterm score.

Grading breakdown

- Discussion Attendance: 10 pts
- Homework: 36 pts
 - Try out questions as many times as you'd like, and get instant feedback as you work
 - Due on **Wednesdays** every week.
 - Note: HW1, HW2, and HW3 are significantly longer than other homeworks.
 - Submitted on [PrairieLearn](#)
- Labs: 24 pts
 - Hands-on experience with course material
 - Due on **Tuesdays** and **Thursdays** every week.
 - You can attend lab sections (twice a week) to work through the lab, or do them on your own
 - Submitted on [Gradescope](#)
- Projects: 80 pts
 - Apply course concepts on larger codebases, 1 partner allowed
 - Submitted on [Gradescope](#)
- Quest: 25 pts
- Midterm: 50 pts
- Final: 75 pts

Class Policies: Collaboration

- Collaboration on assignments
 - Labs and homework: can work in small groups, but must turn in your **OWN** work
 - Projects can either be completed solo or with a partner. If you work with a partner, **all work must be completed synchronously**. (One partner getting help in OH is fine)
- Asking questions and helping others is encouraged
 - Discussing course topics with others is welcome!
- Limits of collaboration
 - Don't share solutions or approaches with each other (except project partners)
 - Golden rule: You should never see or have possession of anyone else's solutions—including from past semesters
 - Be responsible and take precautions (e.g. don't give someone else access to your computer for **any** reason)

Cheating

Please don't cheat...

- Last year, we had >100 cheating cases, mostly from project copying.

Most cases can be categorized into one of three types:

- Accidental cheating (exceeding our limits of allowed collaboration)
 - Most common case
- Rational cheating (deliberately trying to "game" our system)
 - Thankfully very rare
- Irrational cheating (reaching a point of desperation and making a bad decision)
 - Also fairly common

How to avoid Accidental Cheating

Our goal in the class is to maximize the amount of learning that gets done.

General rule: If something can potentially allow someone to get a nonzero score on an assignment without them reaching the assignment's learning objectives, it likely constitutes cheating.

Examples of things that are not allowed:

- Using a friend's solution as a "reference" when writing your own solution
- Using LLMs like ChatGPT or Github Copilot to come up with an algorithm for you
- Working with a non-project partner in a way that you both end up with the same underlying solution to a project.

Examples of things that are allowed:

- High-level discussion (e.g. breaking the main project problem down to ~2-3 smaller subproblems, then individually solving those subproblems)
- "Rubber duck" programming
- Copying/generating snippets of code *if* all you need is the syntax (e.g. computing a^b where a and b are numbers)

Why Cheating is not Rational

Many of our misconduct penalties are designed to deter rational cheating

- If you get caught cheating on an assignment, that's an automatic 0 on that assignment, and a referral to the Center for Student Conduct for university sanctions. Further cheating or a single case of very egregious cheating can lead to harsher penalties.

Our cheat detection methods are very robust

- We have software to detect similarities between all current submissions, submissions from previous semesters, and solutions you can find online
 - LLMs in general have a tendency to respond to project-related questions with code directly from online solutions...
 - Our software checks your entire history, and is not fooled by things like variable renaming and switching around a few lines of code.
- The work required to bypass all our cheat-detection methods is often more than the work to just do the project honestly, so don't cheat.

Even if you bypass all those cheat-detection methods and get points for something that's not your work, you're losing out on the practice we want you to get from the assignment, and will likely lose any points you gained when you do worse on the next exam as a result.

How to avoid Irrational Cheating

Desperation can cause you to do things you would not normally do, even if you know rationally that it'll likely put you in a worse position later.

The best way to avoid irrational cheating is to make sure you never reach that point of desperation in the first place:

- Start projects early! Part of the project is getting stuck and finding a way to get unstuck. Plan enough buffer time in your schedule to make sure you can still finish even if you're stuck for longer than expected.

If you're falling behind in the class, let us know! We'll work with you to maximize the amount of learning you can get from the class.

Class Policies: Academic Misconduct

- We're here to help! There are plenty of staff and resources available for you
 - You can always talk to a staff member if you're feeling stressed or tempted to cheat
- Academic misconduct policies
 - At minimum, the student will receive negative points on the assignment
 - Example: If a project is worth 30 points, the student will receive a score of -30 on the project.
 - The student may be referred to the Center for Student Conduct
 - If you take the class with integrity, you don't need to worry about these!
- Choose your partners wisely!
 - If your project partner cheats, you will be flagged as well on that assignment!
- We check **intermediate student work**, which accounted for ~65% of cases from Spring 2024.
 - **PLAGIARISM AT ANY POINT IN YOUR WORK IS UNACCEPTABLE.**

Stress Management and Mental Health

- We want to reduce your stress where we can

Your well-being is more important than this course

- If you feel overwhelmed, there are options
 - Academically: Ask on Ed, talk to staff in office hours, set up a meeting with staff to make a plan for your success this semester
 - Non-academic:
 - Counseling and Psychological Services (CAPS) has multiple free, confidential services
 - Casual consultations: uhs.berkeley.edu/counseling/lets-talk
 - Crisis management: uhs.berkeley.edu/counseling/urgent
 - Check out UHS's resources: uhs.berkeley.edu/health-topics/mental-health

Stress Management and Mental Health

- Life happens.
 - And sometimes life is really sucky. And that's okay.
 - If you find yourself in a tough situation at any point in the semester, please don't hesitate to reach out! We will *not* judge you in any way; our top priority is just making sure you're supported.
 - We're often our own harshest critics; if you're struggling or are faced with a situation that's affecting your well-being in any way, and can't decide whether to reach out or not, *please do*.
 - There are many ways to handle tough situations from a course-side perspective, ones that don't involve you having to fail a course or perform less than you believe you can!

Class Policies: Extensions and Support

- Deadlines are hard – late work will not receive credit.
 - Reasoning for this: summer moves extremely fast. With an assignment due every day of the week, even missing one can affect your work on the next. If you are falling behind by more than a week, it is suggested to move on and just start with new assignments.
- If you encounter an extenuating circumstance and you need an extension
 - Submit to the student support form. Staff will grant **24-hour extensions in good faith** and **>1 day extensions for severe extenuating circumstances** where you can account for every day missed.
- It is okay to reach out if things come up! We're here to support you!
 - Submit to the student support form.
 - Life happens.
 - You can always reach out for any reason.
 - Feeling lost in the class? Reach out.
 - Stressed out by other classes? Reach out.
 - Not feeling well? Reach out.
- Student support form: <https://cs61c.org/su26/extensions/>

Class Policies: DSP

- Disabled Students' Program (DSP)
 - There's a variety of accommodations UC Berkeley can help us set up for you in this class
 - <https://dsp.berkeley.edu/>
- Are you facing barriers in school due to a disability?
 - Apply to DSP!
 - Only instructors, logistics TAs, and student support TAs can access any DSP-related info
- Our goal is to teach you the material in our course. The more accessible we can make it, the better.

Questions?

Agenda

- Introduction
 - Course Logistics
- **Class Overview**
- Binary
 - Binary and Hexadecimal
 - Representing Data using Binary
 - Binary Mathematics

CS61C is NOT about C Programming...!!

It is about the **hardware-software interface**.

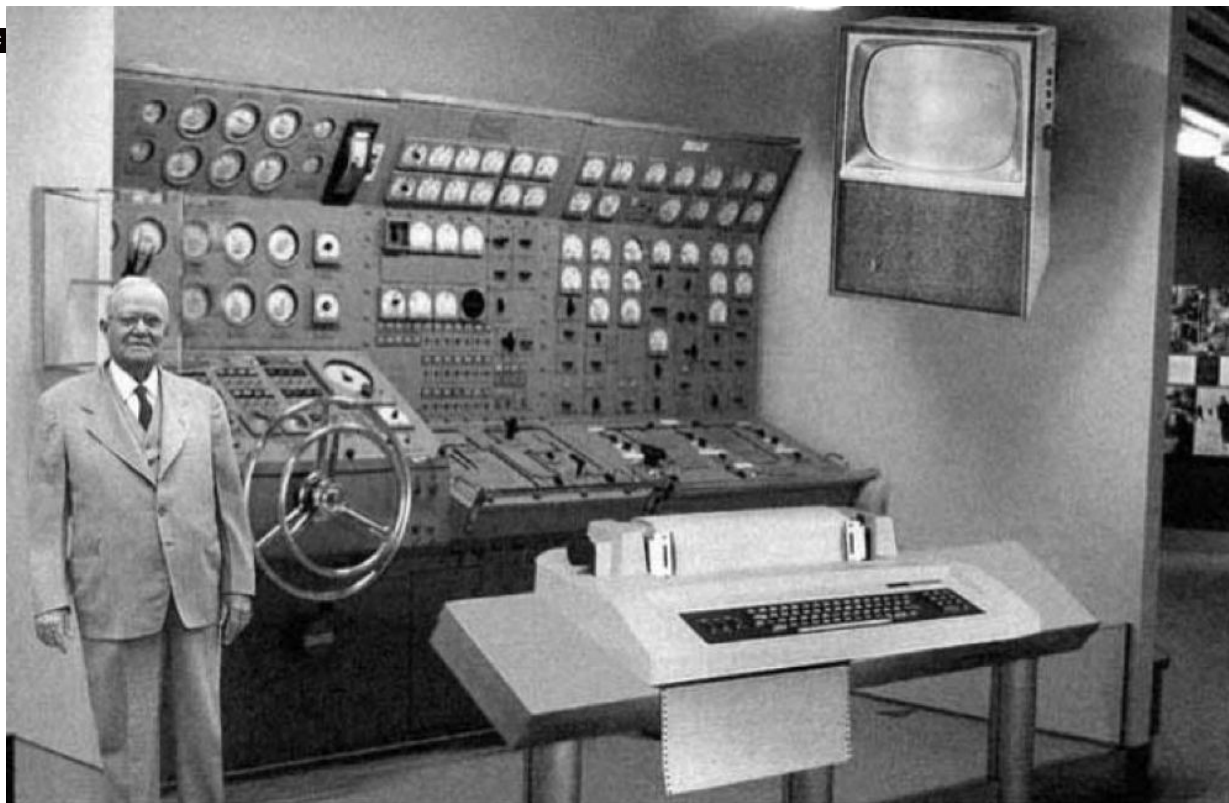
- How a computer is actually built
- How a computer takes the code you write and actually runs it

By the end of this course, we'll have covered all the steps you need to build a computer and run a program on it, given a pile of transistors.

Why is this useful?

- Lets you build/design new devices (start of hardware track).
- Lets you adapt the code you write to the machine it's running on (much more efficient).

Old-School CS61C



Note:
This photo is fake

New-School CS61C (1/3)

Summer 2026



**Personal Mobile
Devices**



**Network Edge
Devices**

New-School CS61C (2/3)

CS 61C

Summer 2026



New-School CS61C (3/3)

CS 61C

Summer 2026



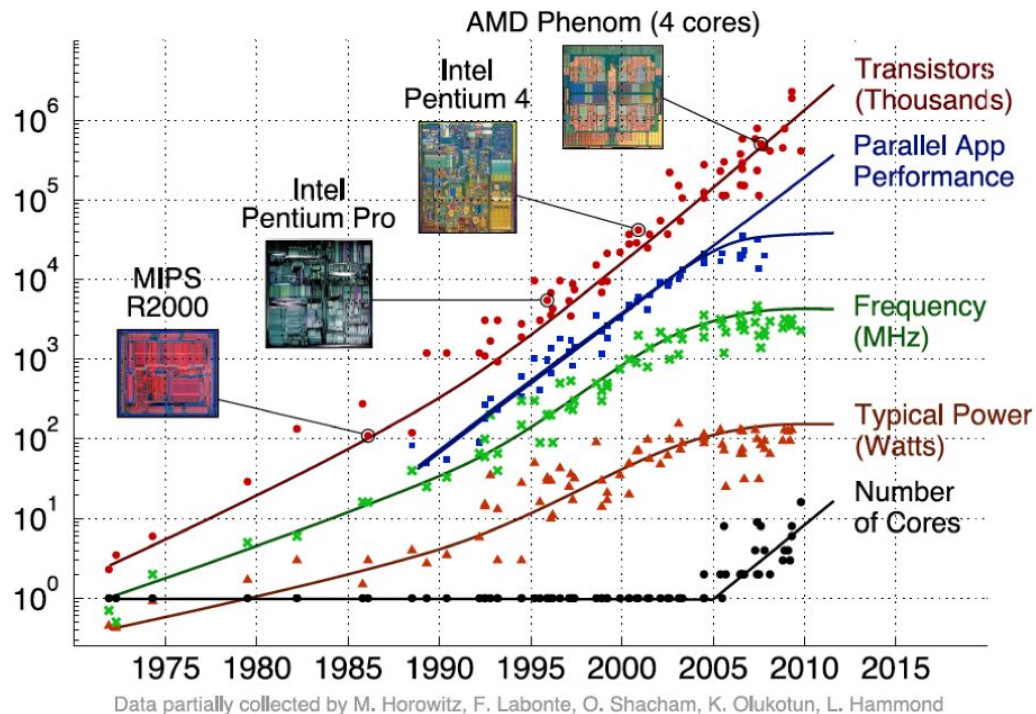
A popular laptop
sticker



Why is computer architecture exciting
today?

Reason 1: Changing Constraints

- Moore's Law ending
- Power limitations
- Amdahl's Law



Reason 2: Era of Domain-Specific Computing

- Number of deployed devices continues to grow, but there is no single killer application.
 - Diversification of needs, architectures
 - Machine learning is common for most domains

Old Conventional Wisdom

- Moore's Law + Dennard Scaling = faster, cheaper, lower-power **general-purpose computers** each year
 - In glory days, 1%/week performance improvement!
- Dumb to compete by designing parallel or specialized computers
 - By time you've finished design, the next generation of general-purpose will beat you!

New Conventional Wisdom

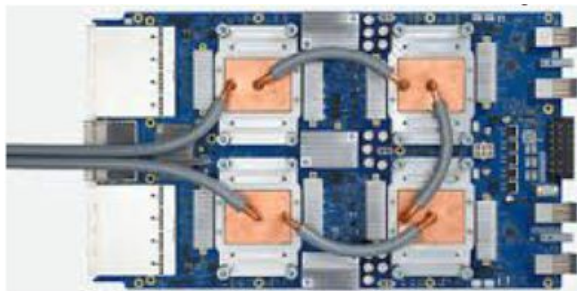
Each domain requires heterogeneous systems.

- Multiple processor cores
- GPUs,
- NPUs,
- accelerators,
- interfaces,
- memory, ...

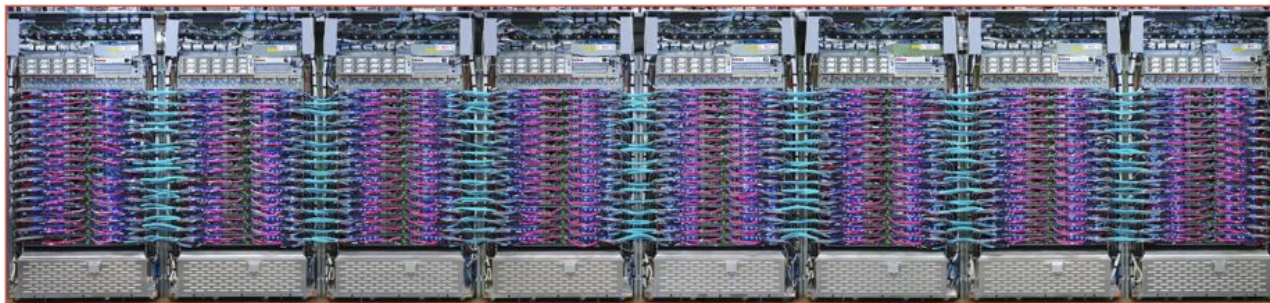


Apple M14 ([link](#))

New Conventional Wisdom



Google TPU3
Specialized Engine for training
Neural Networks
Deployed in cloud



1024 chips, > 100PetaFLOPs

Class Overview

Phase 1: Lower-level programming languages (Weeks 1-3)

- Week 1: C
- Week 2: RISC-V

Phase 2: Hardware (Weeks 3-5)

Phase 3: Optimizations (Weeks 5-7)

Class runs very fast. We assume that you have taken 61A/B or equivalent, and are reasonably comfortable with Python/Java programming and software engineering/debugging.

- LLMs are *terrible* at debugging Logisim circuits. If you've been relying on LLMs to help debug your code, now's the time to learn how to debug on your own.

Agenda

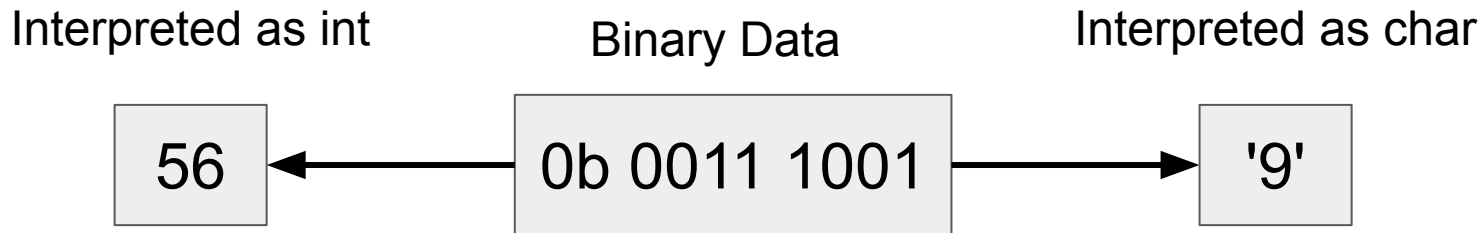
- Introduction
 - Course Logistics
- Class Overview
- **Binary**
 - **Binary and Hexadecimal**
 - Representing Data using Binary
 - Binary Mathematics

Binary

- A system of storing data using just two digits: 1 and 0.
- Everything in a computer is ultimately stored in binary (high voltage wire = 1, low voltage wire = 0), so we need a way to translate our data into binary and back.
- By convention, binary data is written with "0b" as a prefix (and may be split with spaces to make it easier to read)

Important Note: Binary doesn't have meaning

Binary data has no meaning in and of itself; we as humans assign meaning to each bitstring, and read each bitstring as if it was that particular meaning.



Storing Data in Binary

A system to data as binary is known as a **representation scheme**. As long we use a consistent scheme to read and write data, our program will work (even if outside users don't know our scheme).

Ideally, schemes should:

- Represent data that's relevant to whatever we're making
- Store things efficiently
 - Optimal memory use
 - Efficient operators
- Be intuitive for humans to understand
 - Can skip this one if absolutely necessary

Let's take a look at how (unsigned) integers get stored in a computer.

Analogous System: Decimal

- A system of storing numbers using ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.
- Numbers get written using multiple digits, with every integer having a unique representation: If I write “5678”, this means:
- $5 \times 10^3 + 6 \times 10^2 + 7 \times 10^1 + 8 \times 10^0$

Unsigned Integers in Binary

- A system of storing numbers using two digits: 0, 1.
- Numbers get written using multiple digits, with every integer having a unique representation: If I write “0b1010”, this means:
- $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$
- Note: Since we only ever multiply by 1 or 0, we can just write the powers of 2 that have 1s in them:
 - $0b1010 = 2^3 + 2^1$
 - As a corollary, every integer can be uniquely expressed as a sum of powers of 2.

Challenges with using Binary

- (Human) conversion from binary to decimal and back is hard
- Ex. To convert 83 to binary:
 - 83 is greater than $2^6 = 64$, but less than $2^7 = 128$
 - $83 = 1 \times 2^6 + 19$
 - $83 = 2^6 + 2^4 + 3$
 - $83 = 2^6 + 2^4 + 2^1 + 1$
 - $83 = 2^6 + 2^4 + 2^1 + 2^0$
 - $83 = 0b1010011$
- Useful to memorize common binary-to-decimal conversions:
 - Numbers 0 through 15
 - Select powers of 2
 - Approximations for various powers of 2

Useful Numbers to Memorize

Powers of 2

2^1	2	2^9	512
2^2	4	2^{10}	1024
2^3	8	2^{11}	2048
2^4	16	2^{12}	4096
2^5	32	2^{13}	8192
2^6	64	2^{14}	16384
2^7	128	2^{15}	32768
2^8	256	2^{16}	65536

Decimal	Binary
0	0b0000
1	0b0001
2	0b0010
3	0b0011
4	0b0100
5	0b0101
6	0b0110
7	0b0111

Decimal	Binary
8	0b1000
9	0b1001
10	0b1010
11	0b1011
12	0b1100
13	0b1101
14	0b1110
15	0b1111

Binary SI-prefixes

- Note: $2^{10} = 1024$ is extremely close to 1000. There's absolutely no good reason for this; the numbers just happened to line up nicely
- On the plus side, this means that we can approximate large powers of 2 in decimal
 - Ex. $2^{32} = 2^{30} \times 2^2$
 - $= (2^{10})^3 \times 4$
 - $\approx 4 \times (1000)^3$
 - ≈ 4 billion
 - Actual value is 4,294,967,296, ~7% off
- This also gets used in binary SI prefixes
- “Kibi, Mebi, Gibi”... instead of “Kilo, Mega, Giga”
 - Ex. 512 TiB = 512 Tebibytes = 2^{49} bytes.

Binary

- A system of storing data using just two digits: 1 and 0.
- Everything in a computer is ultimately stored in binary (high voltage wire = 1, low voltage wire = 0), so we need a way to translate our data into binary and back.
- By convention, binary data is written with "0b" as a prefix (and may be split with spaces to make it easier to read)
- Generally too unwieldy to write directly, so binary data is often written using hexadecimal or octal
 - Binary numbers use almost 3 times as many digits as decimal, and it's painful to read long binary strings, such as: 0b0101 0000 0110 0001 0110 1001 0110 1110

Hexadecimal

- A system of storing numbers using 16 digits: 0123456789ABCDEF
- By conventions uses "0x" as a prefix to denote hexadecimal data
- Numbers get written using multiple digits, with every integer having a unique representation: If I write "0x87", this means:
- $8 \times 16^1 + 7 \times 16^0$

Converting Hexadecimal to Binary

- Base 16 is chosen because $16 = 2^4$
- Translation between binary and hex is much easier
 - $0x87 = 8 \times 16^1 + 7 \times 16^0$
 - $= 0b1000 \times 2^4 + 0b0111 \times 2^0$
 - $= 0b1000\ 0000 + 0b0111$
 - $= 0b1000\ 0111$
- Conclusion: To convert from hexadecimal to binary:
 - Convert each digit separately to binary
 - Concatenate the results
- To convert from binary to hexadecimal
 - Split the data into sets of 4 digits (prepending 0s to get a multiple of 4 digits)
 - Convert each set of 4 digits into its corresponding hex digit
- Octal is Base 8, which does the same, but using 3 bits at a time instead.

Hexadecimal

- In this course, hex is ALWAYS used as a proxy for binary; we write a hex value for brevity and readability, but the intent is that of the underlying binary.
- All other bases not mentioned don't have a specific prefix, but instead have the base written as a subscript.
 - Ex. $0x14 == 0b0001\ 0100 == 20$
 - Ex. 43_5 (or 43_5) $== 4 \times 5^1 + 3 \times 5^0 == 23$
- Hex to decimal or decimal to hex is about as difficult as binary to decimal.

Agenda

- Introduction
 - Course Logistics
- Class Overview
- **Binary**
 - Binary and Hexadecimal
 - **Representing Data using Binary**
 - Binary Mathematics

Representing Nonnegative Integers

- Problem: There's infinitely many integers. Making a representation that could represent any integer would require infinitely many bits.
- Solution 1: Treat an integer as an array of digits (extend array as needed)
 - Ex. Decimal numbers in math
 - Ex. Python's large integer primitive
 - Would require variable amounts of time to compute operations
- Solution 2: Only allow for numbers up to a max value (fixed bitlength)
 - Ex. C's integer primitive
 - Requires edge case handling when math results exceed the maximum value
 - Often referred to as an n -bit unsigned integer, where we allow numbers from 0 to $2^n - 1$ (ex. 8-bit unsigned int goes from 0 to 255)
 - To write an n -bit number, we always write out the full n bits, including leading zeros

Boolean Values

- Only two possible states: TRUE and FALSE
- Can be stored in 1 bit
 - Assign “1” to mean TRUE, and “0” to mean FALSE
- Operators: OR, AND, NOT, XOR

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

x	NOT x
0	1
1	0

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

Agenda

- Introduction
 - Course Logistics
- Class Overview
- **Binary**
 - Binary and Hexadecimal
 - Representing Data using Binary
 - **Binary Mathematics**

Bitwise Operations

Bitwise operations are operations designed to work directly with bits:

In C, they're written as `&` `|` `~` `^` `<<` `>>`

- The first four correspond to the logical operators AND, OR, NOT, and XOR.
 - There's no real decimal meaning to these operations, but they tend to be useful when working with raw binary data.
 - Note that C defines `^` as XOR, not exponentiation!
- To run these operations:
 - Convert the numbers to binary
 - Run the corresponding logical operator on each pair of bits (1=TRUE, 0=FALSE)

Examples: Bitwise Operations

- Let's assume we're working with 4-bit integers. Compute:
 - $6 \& 5$
 - $6 | 5$
 - $6 \wedge 5$
 - ~ 6

Examples: Bitwise Operations

- Let's assume we're working with 4-bit integers. Compute:

- 6 & 5
- 6 | 5
- 6 ^ 5
- ~6

$$6 = 0b\ 0\ 1\ 1\ 0$$

$$5 = 0b\ 0\ 1\ 0\ 1$$

$$6 \& 5 = 0b\ 0\ 1\ 0\ 0 = 4$$

$$6 | 5 = 0b\ 0\ 1\ 1\ 1 = 7$$

$$6 \wedge 5 = 0b\ 0\ 0\ 1\ 1 = 3$$

$$\sim 6 = 0b\ 1\ 0\ 0\ 1 = 9$$

Bitwise Operations

Bitwise operations are operations designed to work directly with bits:

In C, they're written as `&` `|` `~` `^` `<<` `>>`

- The first four correspond to the logical operators AND, OR, NOT, and XOR.
 - There's no real decimal meaning to these operations, but they tend to be useful when working with raw binary data.
 - Note that C defines `^` as XOR, not exponentiation!
- To run these operations:
 - Convert the numbers to binary
 - Run the corresponding logical operator on each pair of bits (1=TRUE, 0=FALSE)
- **`<<` and `>>` are left shift and right shift, respectively**
 - Left shift: Convert to binary, then move all bits left, appending 0s as needed
 - Right shift (logical): Convert to binary, then move all bits right, prepending 0s as needed

Shifts

- Ex. $0b1011\ 0100 \ll 3$
- Ex. $0b1011\ 0100 \gg 2$

0	0	0	1	0	1	1	0	1	0	0	0	0	0b1011 0100
4	⊖	4	1	0	1	0	0	0	0	0	0	0	0b1010 0000
0	0	0	0	0	1	0	1	1	0	1	⊖	⊖	0b0010 1101

Decimal Analogue: Shifts

- For this slide, we'll use $\ll_{10}$ and $\gg_{10}$ to refer to decimal shifts instead of our normal binary shifts.
- $1234 \ll_{10} 1$ means:
 - Move all the digits one spot left, and add a zero to the end
 - $1234 \rightarrow 12340$
 - Equivalent to multiplying by 10
- $1234 \gg_{10} 1$ means:
 - Move all the digits one spot right, cutting off the last digit
 - $1234 \rightarrow 123$
 - Equivalent to dividing by 10 (rounding down)
- In general:
 - Decimal left shifting by n multiplies a number by 10^n
 - Decimal right shifting by n divides a number by 10^n , rounding down.

Bitwise Operations

- C gives access to the following bitwise operations: `&`, `|`, `~`, `^`, `<<`, `>>`
- The first four correspond to the logical operators AND, OR, NOT, and XOR.
 - There's no real decimal meaning to these operations, but they tend to be useful when working with raw binary data.
 - Note that C defines `^` as XOR, not exponentiation!
- To run these operations:
 - Convert the numbers to binary
 - Run the corresponding logical operator on each pair of bits (1=TRUE, 0=FALSE)
- `<<` and `>>` are left shift and right shift, respectively
 - Left shift: Convert to binary, then move all bits left, appending 0s as needed
 - Right shift (logical): Convert to binary, then move all bits right, prepending 0s as needed
 - Left shift: Equivalent to multiplying by a power of 2
 - Right shift (logical): Equivalent to dividing by a power of 2

Mathematical Operations

- C also allows for mathematical operators: $+$, $-$, $*$, $/$, $\%$, $<$, $>$, $==$, etc.
 - These operators are designed to match the behavior of decimal math as much as possible, so it's always possible to do math with these operators by first converting to decimal, doing the operation, then converting back.
 - Since it takes time to convert to decimal, it's often easier to do the math entirely in binary.
 - The general approach for these is to follow a “grade-school algorithm”, but in binary instead of decimal.

Example: Addition

Decimal

$$\begin{array}{r} 1 \\ 31415 \\ +27128 \\ \hline 58543 \end{array}$$

Binary

$$\begin{array}{r} 11 \\ 0b1001 \\ +0b0011 \\ \hline 0b1100 \end{array}$$

Example: Multiplication

Decimal

$$\begin{array}{r} 31415 \\ \times 123 \\ \hline 94245 \\ 62830 \\ 31415 \\ \hline 3864045 \end{array}$$

Binary

$$\begin{array}{r} 0b001001 \\ \times 0b000011 \\ \hline 0b \quad 1001 \\ 0b \quad 1001 \\ \hline 0b011011 \end{array}$$

Mathematical Operations

- C also allows for mathematical operators: $+$, $-$, $*$, $/$, $\%$, $<$, $>$, $==$, etc.
- Problem: We only can represent a finite number of integers
 - Need to handle cases where the result of “correct” calculation yields a number outside range
- Three main cases (examples below using 8-bit unsigned integer):
 - Going too high (overflow)
 - $200 + 200$
 - $100 * 4$
 - $100 << 2$
 - Going too low (also called overflow)
 - $100 - 200$
 - Fractional result
 - $10 / 3$

Mathematical Operations

- For fractional results: Treat division as floor division (similar to the `//` operator in Python)
 - `10 / 3 == 3`
- For numbers too big or too small: “wrap around” so that the largest number representable + 1 becomes the smallest number
 - Ex. In an 8-bit unsigned integer, `255+1 == 0`
- Equivalent definition: Take the result mod 2^n , or take only the lowest n bits of the number.
 - `200 + 200` is 400, which is $(400-256 = 144) \bmod 256$. Thus, `200 + 200 == 144`
 - `100 * 4 == 144`
 - `100 << 2 == 144`. Note that `100 == 0b0110 0100`, and `144 == 0b1001 0000`, matching the left shift behavior described earlier
 - `100 - 200 == -100`, which is $(-100+256 = 156) \bmod 256$. Thus, `100 - 200 == 156`
 - Note that all of the above values are correct only for math with 8-bit unsigned integers